VELO: Zero-Knowledge Attestation of Forensic Verdicts on a Dual-Ledger Blockchain

Anna Tchijova[1], Olga Vasilieva[1], Dahgoth[1]

[1]*Independent Research — VELO Project · github.com/annatchijova/velo · Apache License 2.0*

Abstract — *A forensic expert who wants a verdict trusted today faces two options, and both are bad: publish the raw evidence — exposing a victim, a private conversation, or a seized device to everyone in the process who did not need to see it — or publish nothing and ask a court, a regulator, or the public to take the expert's word for it. Every production digital-forensics workflow we are aware of picks one of these. This paper presents VELO, a system that separates what must be provable from what must be private, using a deterministic forensic-analysis engine, a hash-chained custody log, and a zero-knowledge circuit deployed on Midnight, a blockchain with an explicit public/private ledger split. The circuit enforces a Daubert-inspired admissibility rule — a verdict of* MALICE *cannot be attested without at least two independently-provenanced corroborating sources — as a cryptographic constraint rather than a policy convention: an attestation that violates it cannot be produced at all. We describe the architecture, report six rounds of adversarial red-teaming against a live deployment on Midnight's* preview *network — including two real, on-chain* MALICE *attestations — and state precisely, rather than by omission, what the resulting proof does and does not establish about the analysis behind it.*

Keywords — zero-knowledge proofs, digital forensics, chain of custody, Daubert standard, verifiable credentials, Midnight, dual-ledger blockchains, anonymous accreditation

1. Introduction

Digital forensic evidence occupies an uncomfortable epistemic position. The only way to fully justify a conclusion built on it is to show the evidence itself — and showing it is frequently the harm the entire process exists to prevent. A memory image from an abuse case contains the abuse material. A fraud investigation's log bundle contains private financial records of people who are not parties to the case. A leak investigation's evidence is, definitionally, the thing that was not supposed to be published. In each of these, the verdict is the deliverable a court or the public actually needs; the evidence is a liability that happens to travel with it.

Two responses to this are in production use today, and neither resolves the tension. The first is full disclosure: publish or hand over the raw evidence so the verdict can be checked. This re-exposes the victim to every reviewer, clerk, or opposing party who did not need to see the material to evaluate whether the *process* that produced the verdict was sound. The second is authority without evidence: a signed report, a credential, an expert's word. This asks the reader to trust a person rather than verify a process — precisely the failure mode admissibility standards such as Daubert[1] were written to close for expert testimony in general, and one digital forensics has so far addressed only by choosing one of the two bad options above.

VELO removes the tradeoff rather than picking a side of it. A deterministic engine produces a verdict from evidence that never leaves the examiner's machine; the verdict, a cryptographic commitment to the analysis, and a zero-knowledge proof that an admissibility rule was satisfied are the only things that reach a public, immutable ledger. Anyone — a judge, opposing counsel, an auditor, the public — can verify that the rule held, without any of them ever holding the evidence that triggered it.

This paper makes the following contributions:

- A deterministic forensic-analysis engine with a fixed four-state verdict scale (`NOISE/SUSPICION/MALICE/ABSTAIN`) computed in exact rational arithmetic, so a threshold comparison gives the same answer on every machine, forever (§3.2).
- A canonical sealing scheme that separates a *bundle hash* (identifies this specific sealing event) from an *analysis fingerprint* (identifies the analysis itself, invariant under re-sealing) over a hash-chained custody log (§3.3).
- A Compact[4] zero-knowledge circuit, deployed on Midnight's *preview* network, that enforces a Daubert-inspired corroboration gate as a circuit constraint — not a convention an application layer could skip — together with a replay guard verified against the live contract (§3.4, §4).
- Two further zero-knowledge layers: an anonymous accredited-expert credential that proves licensure without identifying the examiner, and a commit-reveal blind-second-opinion protocol that lets two examiners agree on a verdict without either seeing the other's opinion first (§3.5–3.6), both demonstrated on-chain.
- Six rounds of adversarial red-teaming, and an explicit, itemized account of what the resulting proof does *not* establish — because a system built specifically to avoid

overclaiming has to start by not overclaiming about itself (§5).

2. Background

2.1 The disclosure dilemma and the Daubert standard

The Daubert standard[1] asks whether an expert's method is testable, has a known error rate, has been subject to peer review, and is generally accepted — a test for the *process* behind a conclusion, not the conclusion's plausibility. A structural requirement that follows from it in practice is corroboration: a finding resting on one source collapses if that source is wrong or fabricated, so a serious verdict conventionally requires independent support from more than one. Jurisdictions outside the United States apply doctrinally similar requirements without a single codified rule of the same name. Digital forensics has generally satisfied this by disclosure — the evidence itself is the only artifact detailed enough for a second expert to audit the reasoning — which is exactly what collides with cases where the evidence cannot be shared.

2.2 Zero-knowledge proofs as a structural solution

A zero-knowledge proof[2] lets a prover convince a verifier that a statement about some private data is true, without revealing the data. Applied here, the statement is not "the evidence shows abuse" — a zero-knowledge circuit does not adjudicate forensic conclusions — but a narrower, sufficient one: *the published verdict is bound to a specific analysis, and if that verdict is the most severe category, at least two independently-provenanced sources support it.* This is the precise sense in which VELO's claims are structural rather than evidentiary: the circuit does not decide whether the forensic conclusion is correct, only that a stated procedural constraint on how the conclusion may be published was satisfied.

2.3 Midnight's dual-ledger model

Midnight exposes an explicit public/private ledger split at the language level: a Compact[4] contract's `disclose()` construct is the only path by which a value computed from private *witness* data may cross into the public ledger, and the compiler — not a convention — enforces that nothing else does. This is the specific primitive VELO's architecture depends on: a chain where "public" and "private" are a boundary the language draws, not a policy the application layer promises to respect. Alternative zero-knowledge platforms were considered and rejected for this reason specifically — Aztec targets general smart-contract privacy without an equivalently explicit dual-ledger split, and Aleo is private by default with disclosure as the exception, the inverse of VELO's requirement that the verdict be public and only the witness be private.

3. System Architecture

3.1 Threat model: two ledgers, one boundary

The design principle is a single boundary, enforced once, rather than many ad hoc redactions: everything an examiner's machine computes is private by default, and only a small, named set of values is deliberately disclosed.

	Public (on-chain ledger)	**Private (witness, examiner's machine)**
Contains	`commitment`, declared `verdict`, attestation count	bundle fingerprint, custody tip, corroboration count, secret salt
Who sees it	Anyone, forever, immutable	Nobody but the examiner; never transmitted
Proves	An attestation exists and follows the rules	Nothing directly — it is the proof's input

Table 1. The compiler-enforced boundary between what the ledger records and what never leaves the examiner's machine.

3.2 The deterministic engine and verdict scale

A small set of detectors runs over evidence — disk images, logs, captures — on the examiner's own machine, and produces a verdict on a fixed four-state scale: `NOISE`, `SUSPICION`, `MALICE`, or `ABSTAIN`. Every comparison inside the decision path uses exact rational arithmetic — never a native float — so the same evidence produces the same verdict on any machine, indefinitely; this is a precondition for a zero-knowledge proof about the analysis to mean anything at all, since a proof about a non-reproducible computation proves nothing reproducible. A `devil_advocate` field is mandatory on any `MALICE` verdict — the strongest innocent explanation considered and why it was rejected — and an empty field blocks sealing.

3.3 Canonical sealing and the custody chain

The verdict, score, which detectors fired, and a custody chain are canonicalized — type-tagged, sorted keys, no floats, a versioned encoder — and hashed into a sealed bundle. Two distinct hashes are kept deliberately apart: a *bundle hash* that includes the sealing timestamp, identifying this specific sealing event, and an *analysis fingerprint* that excludes it, identifying the analysis itself so that a deterministic replay reproduces the same fingerprint. The fingerprint and the custody chain's tip — not the raw bundle — are among the values folded into the on-chain commitment: the fingerprint so a later replay of the same analysis still matches, the tip so the custody history is anchored to something published rather than left mutable on disk.

3.4 The Compact circuit: the Daubert gate as a constraint

The contract publishes `commitment = hash(domain, fingerprint, custody_tip, verdict, corroboration_count, salt)` — every value the attestation asserts is folded into the hash — and enforces the corroboration

rule inside the circuit itself: a `MALICE` verdict requires `corroboration_count >= 2`, checked by an `assert` the prover cannot satisfy otherwise. We confirmed this adversarially against the deployed contract, submitting a hand-crafted single-source witness directly to the circuit, bypassing the engine and every application-level check:

```
failed assert: MALICE requires at least 2
independent corroborating sources — the
Daubert gate
```

No proof is produced under that condition, so nothing reaches the ledger. The guarantee is cryptographic — a property of what the circuit will and will not accept — rather than a promise made by application code that a different code path could bypass.

3.5 Anonymous accredited-expert credentials (Layer 6)

A second contract, `velo_perito`, lets an examiner prove membership in an accredited set and current (non-lapsed) licensure without revealing which member they are. `registerCredential` and `proveCredential` were exercised on Midnight *preview*: an accredited, currently-valid examiner attested a case without disclosing identity, while a credential with a documented licensing gap was refused before any proof could be produced.

3.6 Blind second opinion (Layer 7)

A commit-reveal protocol with a per-`(examiner, case)` nullifier lets two independent examiners each commit a hidden verdict on the same evidence root, then reveal — enforcing that neither could see the other's opinion before both had committed, and that the same examiner cannot vote twice on one case. Demonstrated live on *preview*: two distinct examiners independently committed and revealed `MALICE` on the same Layer-2 evidence Merkle root, reaching `AGREE` without either identity or either interim opinion becoming visible to the other party.

4. Adversarial Evaluation

The methodology throughout is adversarial-first: a claim is not accepted because the intended path succeeds, but because the system was specifically pushed to fail and either refused correctly or the failure was named as a limitation rather than hidden. Six rounds of red-teaming are recorded in full (`docs/RED_TEAM_ROUND_1` through `_6.md`), covering the engine, the sealing layer, the contract, and the deployment scripts.

4.1 Refusal 1 — insufficient corroboration

Evidence engineered to score high enough for `MALICE` but traceable to a single acquisition was submitted end-to-end:

```
[ATTEMPTED VERDICT] SUSPICION
[WHY] Score 0.3000 above the noise ceiling
      but below the MALICE threshold.
Correctly refused. The Daubert corroboration
gate held — this is not a promise, it's a
constraint.
```

4.2 Refusal 2 — broken chain of custody

The byte-for-byte same artifacts that had produced `MALICE`, with only the acquisition record removed, were resubmitted:

```
[VERDICT WITH CUSTODY]     MALICE
[VERDICT WITHOUT CUSTODY]  ABSTAIN
Identical evidence, opposite outcome.
Admissibility is a property of the process,
not of how incriminating the evidence looks.
```

A finding nobody can trace to a lawful acquisition is not, in this design, a weaker finding — it is an inadmissible one, and the system is built to treat the two as the same thing rather than let strong-looking content override a broken process.

4.3 Coverage and live-network evidence

289 tests pass across two independently-run suites (173 engine, 116 frontend), counted by the test runners rather than estimated. The contract is deployed and live on Midnight *preview* (address `46cac58c…023d9d`); two real `MALICE` attestations exist on-chain today, each independently re-verifiable by a script sharing no state with the one that wrote them, and by a third-party block explorer under nobody's control. The circuit's replay guard was verified against the live network: re-attesting an already-published analysis is refused by the contract, not merely discouraged by the client.

5. What the Proof Does and Does Not Establish

A system built specifically to avoid overclaiming has an obligation to state its own boundary precisely rather than by omission. In one sentence: *VELO proves that a specific verdict was produced by a specific process, under specified constraints, and that the resulting attestation cannot be altered afterward. It does not replace forensic judgment; it makes forensic judgment auditable.*

Witness provenance (G1, G3). The circuit proves relationships *between the witness values it is handed* — that a verdict is bound to a fingerprint, that a count clears the gate — not that those values describe a real engine run on real evidence. That binding exists today only in the calling code, which is precisely the part a zero-knowledge proof does not cover. Likewise, the corroboration count is supplied by the prover and checked to be `>= 2`, not cryptographically proven independent — independence is analyst-declared, verified off-chain from distinct provenance roots. This is not specific to VELO: it is what a zero-knowledge proof means for any system attesting to real-world facts rather than pure computation, and closing it requires a witness-provenance mechanism — an engine signature, an accreditation credential, or an environment attestation — that does not yet exist.

Metadata leakage (G4, G5). A commitment, the declared verdict, and a timestamp leave by design. Anyone watching the chain learns that an investigation existed, roughly when, and its outcome category — and attestations from one wallet are linkable to each other, revealing an examiner's case volume, verdict distribution, and cadence, though never case content. The anonymous credential (§3.5) proves accreditation without identifying the examiner; the wallet that *submits* the transaction is still linkable, so closing this fully still needs a browser-signed, rotating-address path.

Dependent and unbuilt pieces. There is no rule-version binding yet — the threshold is fixed in the circuit, with no marker of which rule version checked an older attestation — and no revocation model for a lapsed credential, deferred as meaningless to design before the credential it would revoke. The browser-signed attestation path does not exist: attesting today runs through a CLI using a seed-derived wallet on the examiner's own machine, architecturally equivalent but not identical to a wallet-connected UI. The server does not verify that a submitted `devil_advocate` field is anchored to the actual evidence — one prompt-injection attempt against an LLM-driven client was tested and refused, but by the model's judgment, not a server-side check.

What VELO explicitly does not solve. It does not stop an examiner who is dishonest from the start of the analysis. It removes the possibility of post-hoc tampering with an already-sealed verdict and the possibility of an unverifiable claim of process; it does not remove the possibility of a corrupt examiner at the moment of analysis, which remains a human and judicial responsibility exactly as with any forensic report today.

6. Why This Matters Beyond One Tool

The disclosure dilemma this paper opens with is not specific to digital forensics. Any pipeline that produces an adjudicated decision from sensitive material — a medical-malpractice review built on patient records, a content-moderation verdict built on the reported content itself, a credential-verification process built on personal documents — faces the identical binary: publish the sensitive input to justify the output, or ask for trust in the output alone. VELO's pattern — seal first, prove second, disclose the least sufficient statement — is a transferable architecture for that entire class of problem, not a forensics-specific trick.

The economic argument sharpens this beyond ethics. A forensic expert's reputation today is almost entirely social: certificates, testimonials, years of practice, asserted rather than demonstrated. A verifiable, privacy-preserving track record — "143 sealed cases, 98 accepted under the admissibility rule, zero invalidated attestations," provable without revealing a single victim, client, or case file — changes what an expert-witness market can verify about a candidate before hiring them, and what a court can verify about a witness before crediting their testimony. Where a professional profile today says "I claim expertise," a sealed, provable case history says "cryptographic proof exists that I participated in analyses satisfying specific, checkable rules" — a categorically stronger claim, and one that does not require anyone to trust the platform making it, only the proof.

7. Related Work and Provenance

VELO's cryptographic and blockchain layer is original to this project; several of its conceptual patterns are deliberate adaptations, in a different language and a different trust model, from prior work by the same research group. The deterministic verdict scale and the corroboration-count gate adapt threshold logic from VIGÍA[6], a Python deterministic forensic-intentionality engine; the two-hash sealing pattern (bundle identity versus analysis identity) and the hash-chained, anti-truncation custody log adapt the same distinction from VIGÍA's bundle builder and tool-execution-log chain. The confidence-ceiling-by-corroboration-diversity principle adapts from an audit-chain project, *cronos*; the adversarial test taxonomy (forge, truncate, tamper) adapts from a sealing-and-audit project, *forge*; consent-gated selective disclosure and the local-first, loopback-only frontend pattern adapt from a digital-legacy project, *continuum*. None of this is copy-pasted — the prior work is Python, VELO is TypeScript and a zero-knowledge circuit language, and no file from any prior repository was opened for copy-paste during VELO's construction. What transfers is the shape of the solution: which invariant to enforce, not which line of code enforces it.

8. Conclusion and Future Work

VELO demonstrates that the publish-the-evidence-or-ask-for-trust binary is not architecturally necessary: a deterministic engine, a canonical seal, and a zero-knowledge circuit on a dual-ledger chain are sufficient to make a forensic verdict publicly verifiable while its evidence stays sealed, and sufficient to make an admissibility rule a cryptographic constraint rather than a policy that a careless implementation could skip. The system is

deployed and operating against a live network, not a simulation: two real attestations exist on Midnight *preview* today, independently re-verifiable by a third party.

The gap that remains is exactly the one named in §5, not a different one discovered later: proof of witness provenance. Future work is the witness-provenance mechanism itself — binding a proof to a specific, attested engine execution rather than to whatever bytes a caller supplies — together with the browser-signed attestation path, rule-versioning inside the circuit, and a revocation model for the credential built in §3.5. Each is a closable, specific gap rather than an open-ended one, which is itself the point: a system that names its remaining boundary precisely is one whose remaining work can be verified as done, rather than merely claimed.

References

1. *Daubert v. Merrell Dow Pharmaceuticals, Inc.*, 509 U.S. 579 (1993).
2. Goldwasser, S., Micali, S., Rackoff, C. "The Knowledge Complexity of Interactive Proof Systems." *SIAM Journal on Computing*, 18(1), 1989.
3. Midnight Network. Technical documentation. `docs.midnight.network`, 2026.
4. Midnight Network. Compact language reference and compiler, v0.31.1. `docs.midnight.network/relnotes`, 2026.
5. Camenisch, J., Lysyanskaya, A. "An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation." *EUROCRYPT 2001*.
6. Tchijova, A. VIGÍA: A Deterministic Forensic Intentionality Engine for the SANS SIFT Workstation. `github.com/annatchijova/vigia-intent-analysis`, 2026.
7. Tchijova, A., Vasilieva, O., Dahgoth. VELO Red Team Reports, Rounds 1–6. `github.com/annatchijova/velo/docs/RED_TEAM_ROUND_{1..6}.md`, 2026.
8. Tchijova, A., Vasilieva, O., Dahgoth. VELO source repository and live Midnight *preview* deployment. `github.com/annatchijova/velo`, 2026. Contract: `0x46cac58c…023d9d`.